program_1

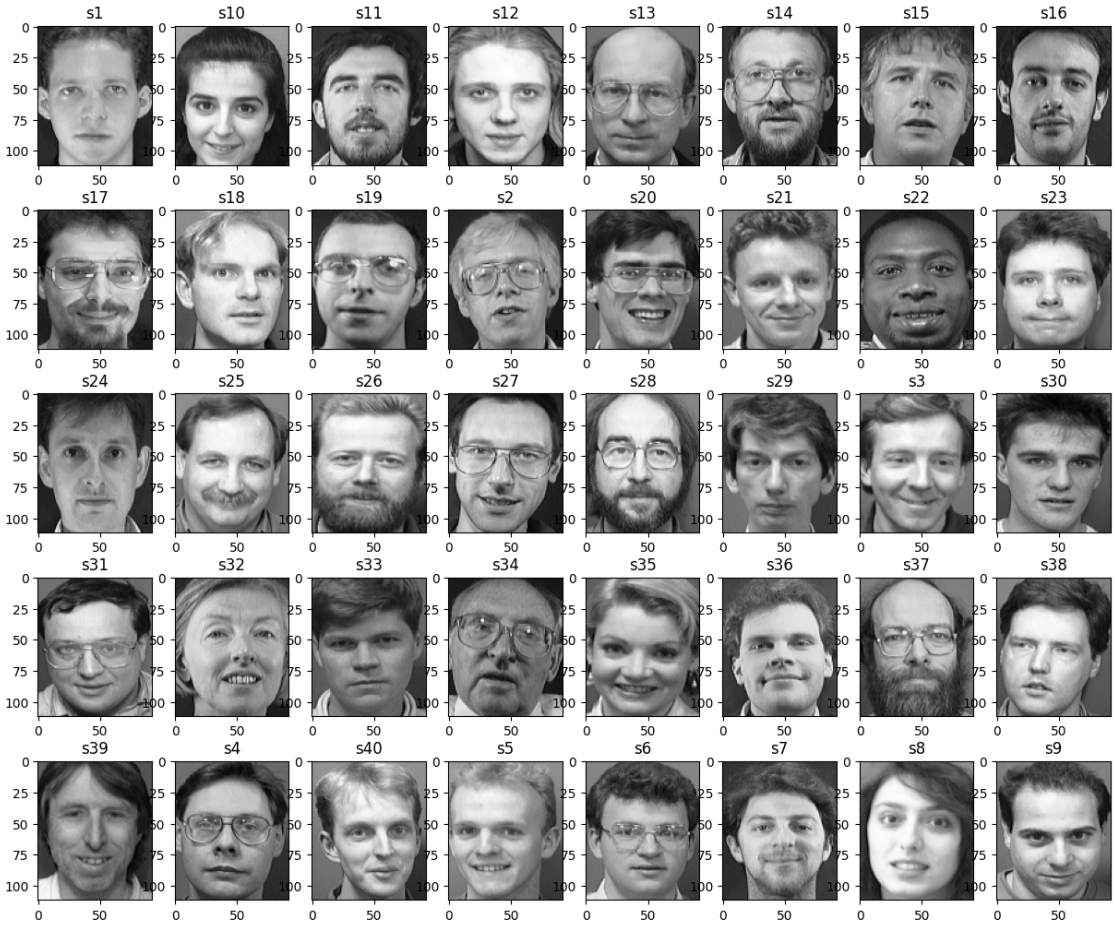
June 26, 2025

Apply LDA for dimensionality reduction and any classifier for face recognition on the ORL dataset and analyze the performance.

[ ]:
```
import os
import cv2
import matplotlib.pyplot as plt
import shutil
import pandas as pd
import numpy as np
import seaborn as sns
# import plotly.express as px
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
from sklearn.svm import SVC


dataset_path = "../datasets/ORL_dataset"
```

[122]:
```
classes = os.listdir(dataset_path)
```

[ ]:
```
plt.subplots(5, 8, figsize=(15, 12.5))
plt.suptitle("Sample Images from ORL Dataset", fontsize=12)
for class_name in classes:
    image = cv2.imread(os.path.join(dataset_path, class_name, "1.pgm"), cv2.
↪IMREAD_GRAYSCALE)
    plt.subplot(5, 8, classes.index(class_name) + 1)
    plt.imshow(image, cmap='gray')
    plt.title(f"{class_name}")
```



[6]:
```
test_ratio = 0.2

classes = os.listdir(dataset_path)

for class_name in classes:

    os.makedirs(f"../datasets/train/{class_name}", exist_ok=True)
    os.makedirs(f"../datasets/test/{class_name}", exist_ok=True)

    images = os.listdir(f"../datasets/ORL_dataset/{class_name}")
    train_images, test_images = train_test_split(images, test_size=test_ratio,␣
 ↪random_state=42)

    for img in train_images:
```

```
        shutil.copy(f"../datasets/ORL_dataset/{class_name}/{img}", f"../datasets/
↪train/{class_name}/{img}")
    print(f"Copied images for {class_name} to train folder")

    for img in test_images:
        shutil.copy(f"../datasets/ORL_dataset/{class_name}/{img}", f"../datasets/
↪test/{class_name}/{img}")
    print(f"Copied images for {class_name} to test folder")
```

```
Copied images for s1 to train folder
Copied images for s1 to test folder
Copied images for s10 to train folder
Copied images for s10 to test folder
Copied images for s11 to train folder
Copied images for s11 to test folder
Copied images for s12 to train folder
Copied images for s12 to test folder
Copied images for s13 to train folder
Copied images for s13 to test folder
Copied images for s14 to train folder
Copied images for s14 to test folder
Copied images for s15 to train folder
Copied images for s15 to test folder
Copied images for s16 to train folder
Copied images for s16 to test folder
Copied images for s17 to train folder
Copied images for s17 to test folder
Copied images for s18 to train folder
Copied images for s18 to test folder
Copied images for s19 to train folder
Copied images for s19 to test folder
Copied images for s2 to train folder
Copied images for s2 to test folder
Copied images for s20 to train folder
Copied images for s20 to test folder
Copied images for s21 to train folder
Copied images for s21 to test folder
Copied images for s22 to train folder
Copied images for s22 to test folder
Copied images for s23 to train folder
Copied images for s23 to test folder
Copied images for s24 to train folder
Copied images for s24 to test folder
Copied images for s25 to train folder
```

```
Copied images for s25 to test folder
Copied images for s26 to train folder
Copied images for s26 to test folder
Copied images for s27 to train folder
Copied images for s27 to test folder
Copied images for s28 to train folder
Copied images for s28 to test folder
Copied images for s29 to train folder
Copied images for s29 to test folder
Copied images for s3 to train folder
Copied images for s3 to test folder
Copied images for s30 to train folder
Copied images for s30 to test folder
Copied images for s31 to train folder
Copied images for s31 to test folder
Copied images for s32 to train folder
Copied images for s32 to test folder
Copied images for s33 to train folder
Copied images for s33 to test folder
Copied images for s34 to train folder
Copied images for s34 to test folder
Copied images for s35 to train folder
Copied images for s35 to test folder
Copied images for s36 to train folder
Copied images for s36 to test folder
Copied images for s37 to train folder
Copied images for s37 to test folder
Copied images for s38 to train folder
Copied images for s38 to test folder
Copied images for s39 to train folder
Copied images for s39 to test folder
Copied images for s4 to train folder
Copied images for s4 to test folder
Copied images for s40 to train folder
Copied images for s40 to test folder
Copied images for s5 to train folder
Copied images for s5 to test folder
Copied images for s6 to train folder
Copied images for s6 to test folder
Copied images for s7 to train folder
Copied images for s7 to test folder
Copied images for s8 to train folder
Copied images for s8 to test folder
Copied images for s9 to train folder
Copied images for s9 to test folder
```

[32]:
```
train_set = "../datasets/train"
test_set = "../datasets/test"
```

[123]:
```
images_X = []
labels_y = []
for class_name in classes:
    class_path = os.path.join(train_set, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        images_X.append(image.flatten())
        labels_y.append(classes.index(class_name))
print(f"Number of images in train set: {len(images_X)}")
```

```
Number of images in train set: 320
```

[58]:
```
labels_y = np.array(labels_y)
images_X = pd.DataFrame(images_X)
```

[128]:
```
lda = LDA(n_components=39)
```

[129]:
```
images_X_lda = lda.fit_transform(images_X, labels_y)
```

[130]:
```
images_X_lda_df = pd.DataFrame(images_X_lda)
```

[131]:
```
images_X_lda_df
```

[131]:

	0	1	2	3	4	5	6	\
0	-4.195820	1.440183	-9.526318	0.985577	-2.238878	2.710791	-3.748485	
1	-5.915632	3.863930	-9.567232	7.083888	-2.749441	1.659635	-1.617893	
2	-7.663957	1.713726	-10.372344	0.992018	-2.778074	-3.278244	-0.597992	
3	-7.147968	-0.097785	-9.072887	4.785691	-0.299194	-0.654152	-1.655769	
4	-7.168803	0.240219	-8.515570	2.347091	0.333050	1.075298	0.154636	
..	...	...	...	...	...	...	...	
315	-0.170546	0.317235	6.252161	2.901864	-2.681779	1.557313	-0.881094	
316	0.060807	2.557154	5.252318	2.988304	-2.362040	0.248275	0.788925	
317	-1.398813	1.105114	7.809580	-0.047862	-3.745474	1.667271	-0.299620	
318	-2.143815	1.692445	7.472940	1.946441	-2.147201	1.074111	0.807094	
319	-0.624043	3.445896	6.086466	2.135989	-3.398222	1.754490	1.134245	

	7	8	9	...	29	30	31	\
0	-2.019688	0.134558	3.244815	...	-0.146298	-0.010709	-0.762054	
1	0.196132	-3.446693	2.177965	...	-0.934556	-0.862796	1.249658	
2	0.730577	-0.502767	2.438865	...	-0.771264	-0.053586	-0.718594	
3	1.404543	-0.908319	4.447840	...	1.408527	0.096570	1.138816	
4	1.959033	-0.608113	4.345680	...	0.538302	1.066564	0.165476	
..	...	...	...	...	...	...	...	
315	-3.414175	4.079278	0.662487	...	-0.219404	0.386893	0.199659	

```
316 -7.416892  2.354512  2.145689  ... -0.030777  0.476900 -0.445431
317 -8.218174  1.227088  1.422216  ... -0.223651  0.550969  0.188937
318 -3.373334  1.865819  1.020588  ... -2.101607  1.308368  0.081416
319 -5.636890  1.547634  2.705579  ... -0.690605 -0.635428 -0.563056

           32        33        34        35        36        37        38
0   -1.603679 -1.450909 -0.717439 -0.158892  1.904321 -0.417238  0.202857
1    2.794213 -0.604517  0.144966 -1.761977  1.138139 -1.043206 -2.155021
2    0.994174  1.035052  0.030680 -2.419612  0.035202  2.448436  0.102956
3    1.260218 -1.183134  0.454599 -2.101472  0.518102 -0.521245 -0.236318
4    1.916962  0.106944 -0.846340 -1.257969  0.971374  0.499374  0.184096
..        ...       ...       ...       ...       ...       ...       ...
315  0.136986 -1.332430 -0.904999  1.379555  1.063660 -0.688705 -0.289642
316  0.168788  1.588798 -0.760137  0.041938 -2.940723 -0.561291 -0.915768
317 -1.548236  0.209080  1.840205 -1.056945  0.294249  0.861716 -2.366692
318 -0.980464 -1.099119 -0.399265 -1.529202 -0.458902  0.489447  1.301076
319  0.472765  0.939752  0.920002  1.099919 -1.967416 -0.346416  1.992791

[320 rows x 39 columns]
```

[134]:
```
test_images_X = []
test_labels_y = []
for class_name in classes:
    class_path = os.path.join(test_set, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        test_images_X.append(image.flatten())
        test_labels_y.append(classes.index(class_name))
```

[135]:
```
test_images_X_df = pd.DataFrame(test_images_X)
test_labels_y = np.array(test_labels_y)
```

[136]:
```
test_images_X_lda = lda.transform(test_images_X_df)
test_images_X_lda_df = pd.DataFrame(test_images_X_lda)
```

Training SVM with LDA applied on the features

[179]:
```
svm_01 = SVC(kernel="linear", gamma="scale")
svm_01.fit(images_X_lda, labels_y)
```

[179]:
```
SVC(kernel='linear')
```

[167]:
```
y_pred = svm_01.predict(test_images_X_lda)
```

[168]:
```
cl_report = classification_report(test_labels_y, y_pred, target_names=classes)
print(cl_report)
```

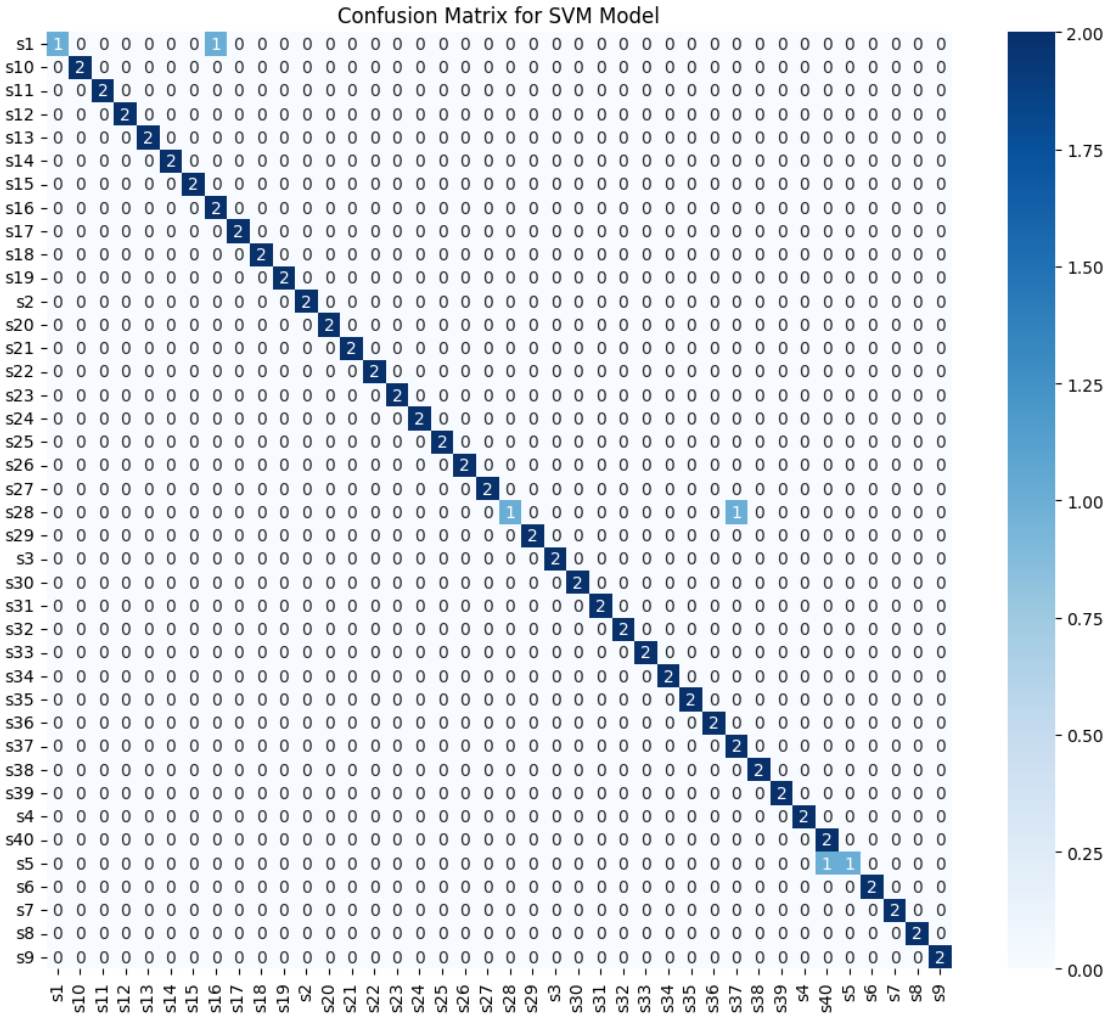
	precision	recall	f1-score	support
s1	1.00	0.50	0.67	2
s10	1.00	1.00	1.00	2
s11	1.00	1.00	1.00	2
s12	1.00	1.00	1.00	2
s13	1.00	1.00	1.00	2
s14	1.00	1.00	1.00	2
s15	1.00	1.00	1.00	2
s16	0.67	1.00	0.80	2
s17	1.00	1.00	1.00	2
s18	1.00	1.00	1.00	2
s19	1.00	1.00	1.00	2
s2	1.00	1.00	1.00	2
s20	1.00	1.00	1.00	2
s21	1.00	1.00	1.00	2
s22	1.00	1.00	1.00	2
s23	1.00	1.00	1.00	2
s24	1.00	1.00	1.00	2
s25	1.00	1.00	1.00	2
s26	1.00	1.00	1.00	2
s27	1.00	1.00	1.00	2
s28	1.00	0.50	0.67	2
s29	1.00	1.00	1.00	2
s3	1.00	1.00	1.00	2
s30	1.00	1.00	1.00	2
s31	1.00	1.00	1.00	2
s32	1.00	1.00	1.00	2
s33	1.00	1.00	1.00	2
s34	1.00	1.00	1.00	2
s35	1.00	1.00	1.00	2
s36	1.00	1.00	1.00	2
s37	0.67	1.00	0.80	2
s38	1.00	1.00	1.00	2
s39	1.00	1.00	1.00	2
s4	1.00	1.00	1.00	2
s40	0.67	1.00	0.80	2
s5	1.00	0.50	0.67	2
s6	1.00	1.00	1.00	2
s7	1.00	1.00	1.00	2
s8	1.00	1.00	1.00	2
s9	1.00	1.00	1.00	2
accuracy			0.96	80
macro avg	0.97	0.96	0.96	80
weighted avg	0.97	0.96	0.96	80

[164]:
```
ac_score = accuracy_score(test_labels_y, y_pred)
print(f"Accuracy of the SVM model on test set: {ac_score}")
```

```
Accuracy of the SVM model on test set: 0.9625
```

[178]:
```
cf = confusion_matrix(test_labels_y, y_pred)
plt.figure(figsize=(12, 10))
sns.heatmap(cf, annot=True, fmt='d', cmap='Blues', xticklabels=classes,␣
 ↪yticklabels=classes)
plt.title("Confusion Matrix for SVM Model")
```

[178]:
```
Text(0.5, 1.0, 'Confusion Matrix for SVM Model')
```



Training SVM with original features

[ ]:
```
svm_02 = SVC(kernel="linear", gamma="scale")
svm_02.fit(images_X, labels_y)
```

```
CPU times: total: 0 ns
Wall time: 0 ns
```

[170]:
```
y_pred_02 = svm_02.predict(test_images_X_df)
cl_report_02 = classification_report(test_labels_y, y_pred_02,␣
 ↪target_names=classes)
print(cl_report_02)
```

```
              precision    recall  f1-score   support

          s1       1.00      1.00      1.00         2
         s10       1.00      0.50      0.67         2
         s11       1.00      1.00      1.00         2
         s12       1.00      1.00      1.00         2
         s13       1.00      1.00      1.00         2
         s14       1.00      1.00      1.00         2
         s15       1.00      1.00      1.00         2
         s16       1.00      1.00      1.00         2
         s17       1.00      1.00      1.00         2
         s18       0.67      1.00      0.80         2
         s19       1.00      1.00      1.00         2
          s2       1.00      1.00      1.00         2
         s20       1.00      1.00      1.00         2
         s21       1.00      1.00      1.00         2
         s22       1.00      1.00      1.00         2
         s23       1.00      1.00      1.00         2
         s24       1.00      1.00      1.00         2
         s25       1.00      1.00      1.00         2
         s26       1.00      1.00      1.00         2
         s27       1.00      1.00      1.00         2
         s28       1.00      0.50      0.67         2
         s29       1.00      1.00      1.00         2
          s3       1.00      1.00      1.00         2
         s30       1.00      1.00      1.00         2
         s31       1.00      1.00      1.00         2
         s32       1.00      1.00      1.00         2
         s33       1.00      1.00      1.00         2
         s34       1.00      1.00      1.00         2
         s35       1.00      1.00      1.00         2
         s36       1.00      1.00      1.00         2
         s37       0.67      1.00      0.80         2
         s38       1.00      1.00      1.00         2
         s39       1.00      1.00      1.00         2
          s4       1.00      1.00      1.00         2
         s40       1.00      1.00      1.00         2
```

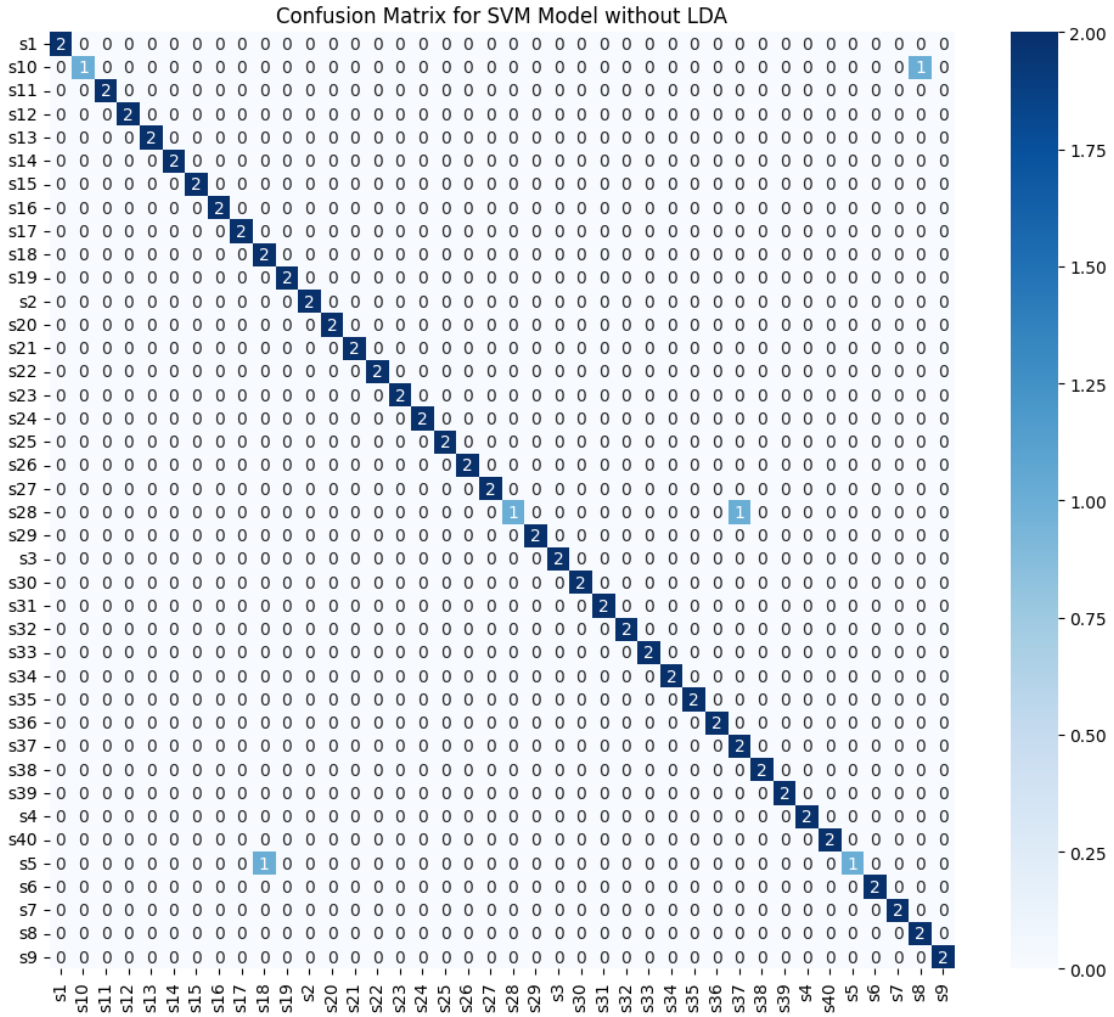
```
           s5       1.00      0.50      0.67         2
           s6       1.00      1.00      1.00         2
           s7       1.00      1.00      1.00         2
           s8       0.67      1.00      0.80         2
           s9       1.00      1.00      1.00         2

     accuracy                           0.96        80
    macro avg       0.97      0.96      0.96        80
 weighted avg       0.97      0.96      0.96        80
```

[171]:
```
ac_score_02 = accuracy_score(test_labels_y, y_pred_02)
print(f"Accuracy of the SVM model on test set without LDA: {ac_score_02}")
```

```
Accuracy of the SVM model on test set without LDA: 0.9625
```

[175]:
```
cf_02 = confusion_matrix(test_labels_y, y_pred_02)
plt.figure(figsize=(12, 10))
sns.heatmap(cf_02, annot=True, fmt='d', cmap='Blues', xticklabels=classes,␣
↪yticklabels=classes)
plt.title("Confusion Matrix for SVM Model without LDA")
```

[175]:
```
Text(0.5, 1.0, 'Confusion Matrix for SVM Model without LDA')
```


Confusion Matrix for SVM Model without LDA
s1 s10 s11 s12 s13 s14 s15 s16 s17 s18 s19 s2 s20 s21 s22 s23 s24 s25 s26 s27 s28 s29 s3 s30 s31 s32 s33 s34 s35 s36 s37 s38 s39 s4 s40 s5 s6 s7 s8 s9
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00